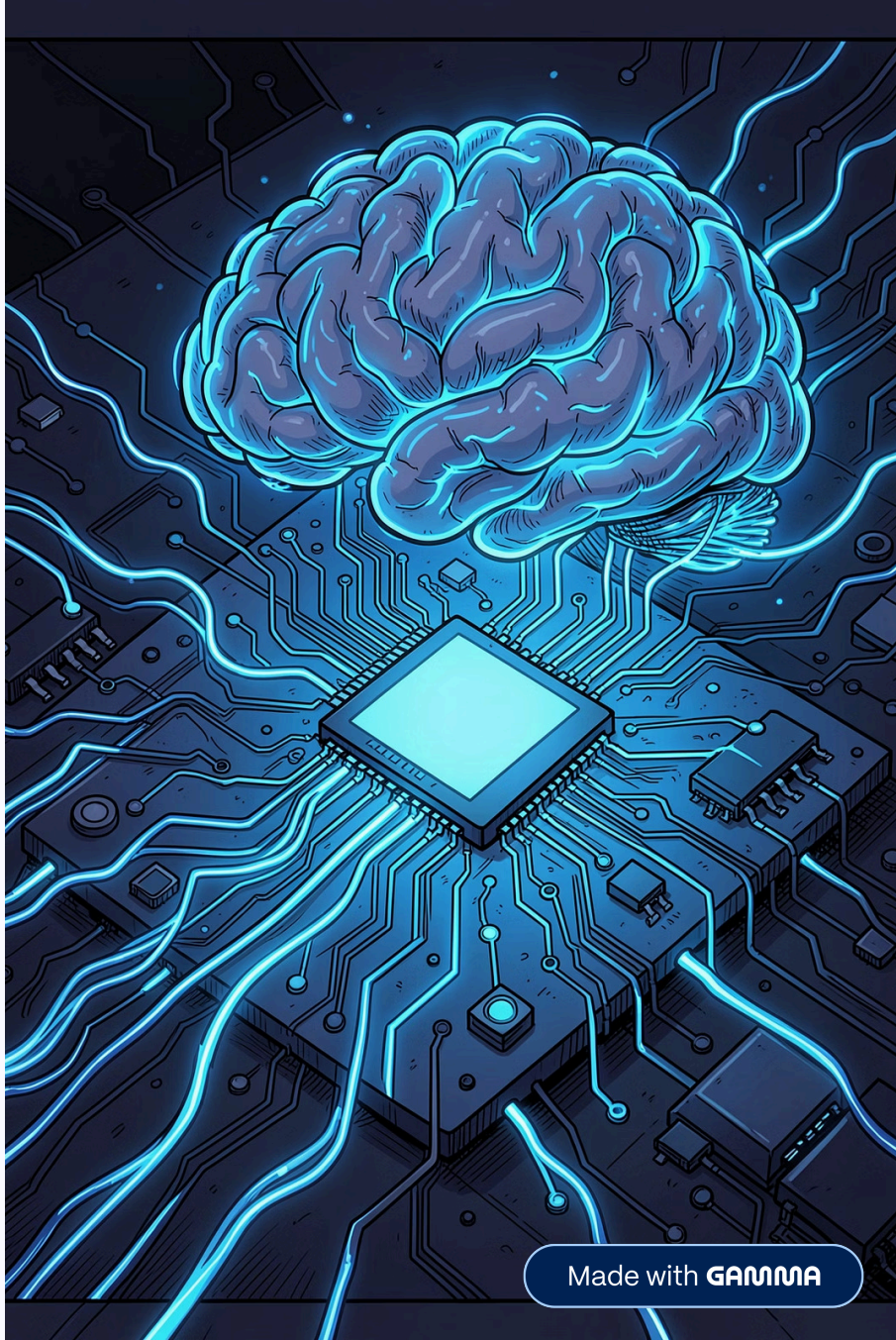
AI Logic & Decision Flow

How autonomous agents generate, evaluate, and commit code – with adversarial oversight at every step.



Where Does AI Intervene?

Local Cognitive Core

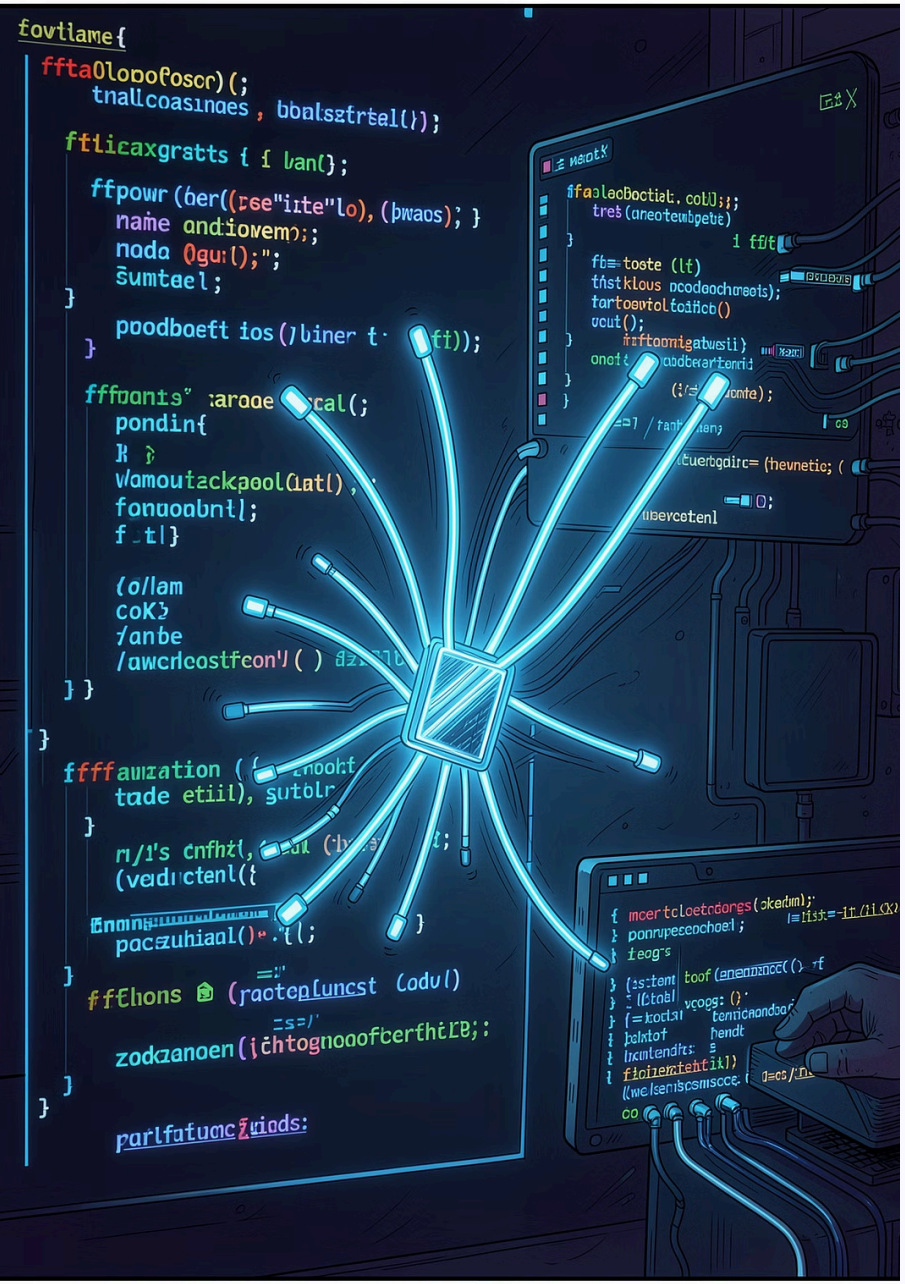
Spawns local **LLM code generators** (qwen2.5-coder via Ollama) to analyze JIRA tasks and write code patches.

Parallel Mutation

Generates three concurrent code mutants at varying temperatures (0.20, 0.70, 0.95) to explore diverse logical approaches.

Adversarial Audit

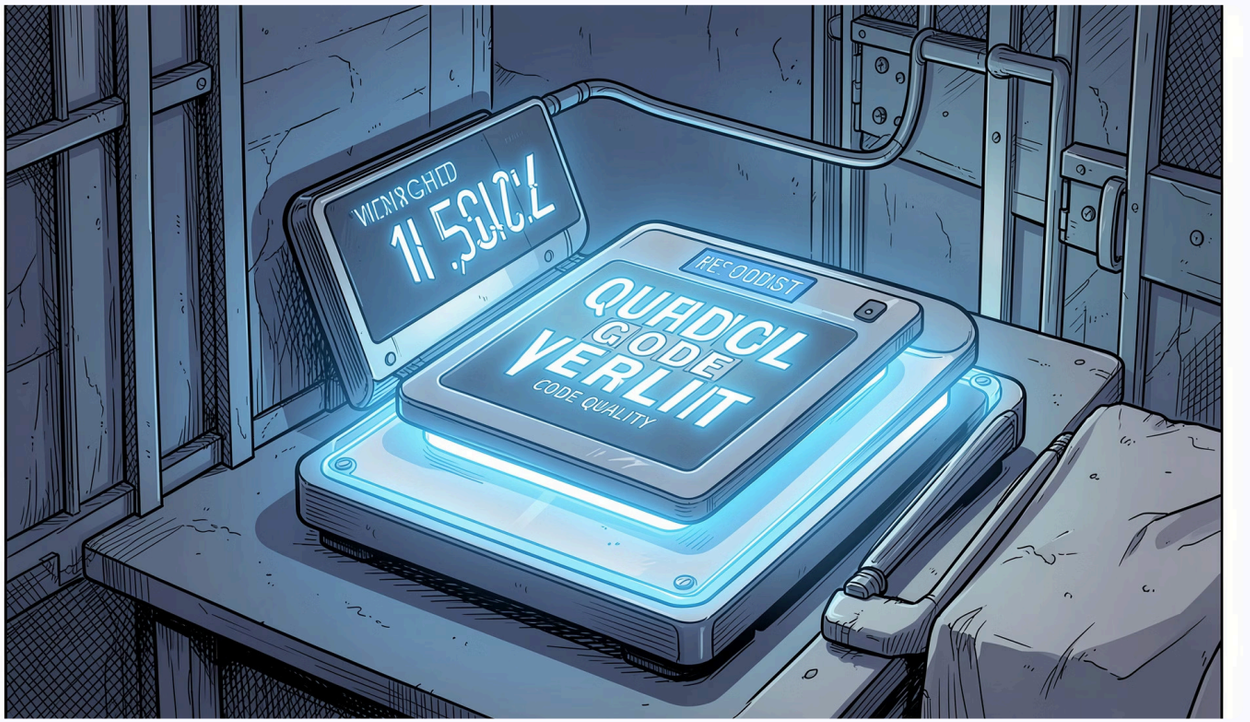
The **critic.py** model acts as an independent reviewer, evaluating code differences, test outputs, and compiler logs.



Parallel Mutation in Action

Three mutants are generated simultaneously at different temperatures – enabling the system to explore a wide range of logical approaches before any code is committed.

How Are Decisions Made?



Adversarial Verdict Gates

Code is committed only if the **Critic's** quantitative evaluation score is **>= 0.88**.

AST-Level Interception

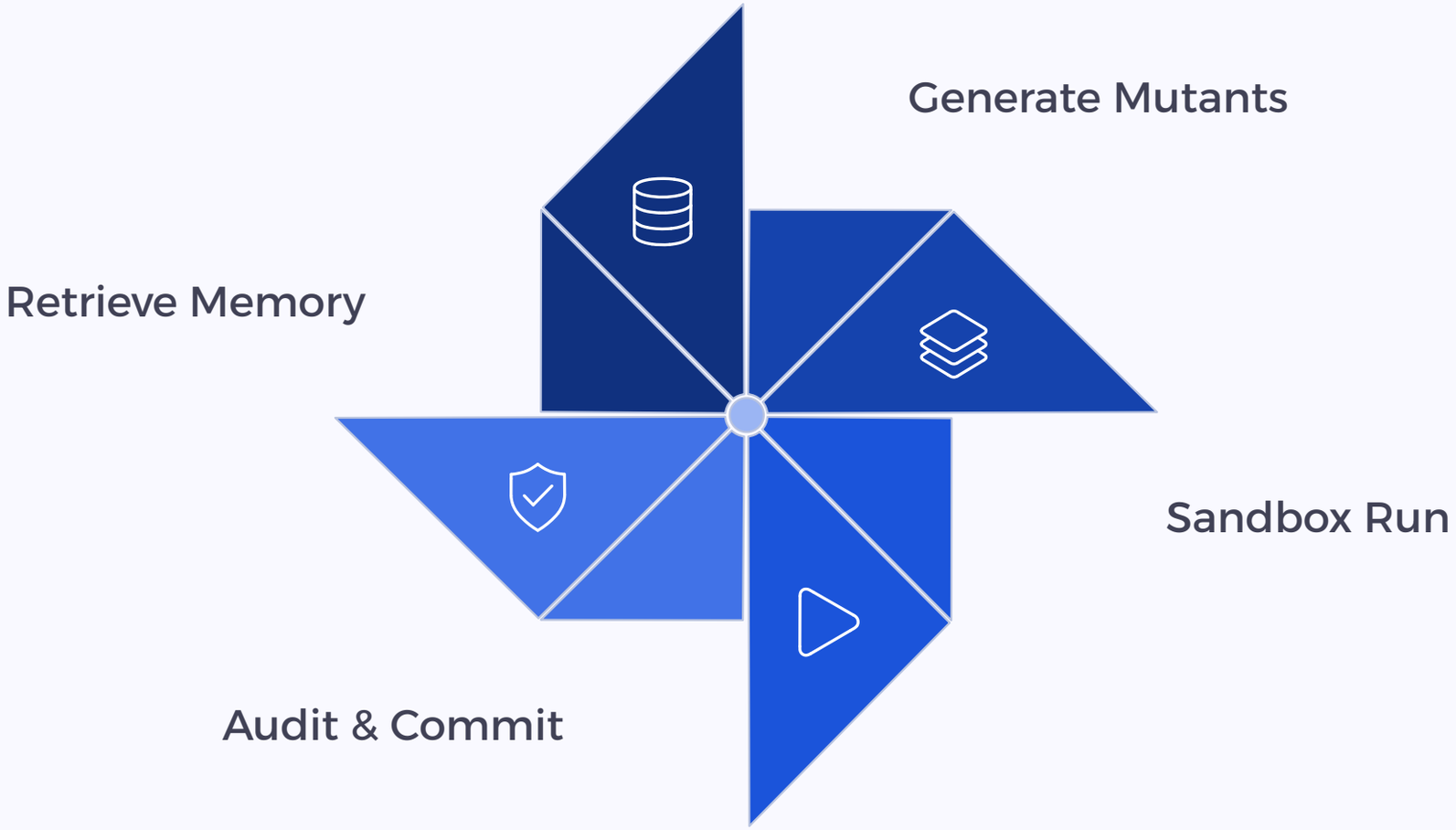
Runaway infinite loops are automatically terminated by the **AST Gas Meter** in under **1 millisecond** if instruction limits are breached.

Goal Drift Guard

The execution loop is aborted and rolled back if the **Goal Drift Index (GDI)** exceeds **0.60**, detecting semantic deviation from the task goal.

The Orchestrator Loop

Manages the step-by-step cycle of every autonomous coding task.



The orchestrator coordinates each phase – from memory retrieval through mutant generation, sandbox execution, adversarial audit, and final commit or rollback – ensuring every step is validated before proceeding.

Collective Memory — ANJANEYA

Vector Crystallization

Successful runs are crystallized as vectors in a shared **LanceDB Manifold**, allowing all agents to reuse successful patterns.

Git Synchronization

Keeps host system state clean via **atomic commits** for passes, and **git rollbacks** (`git checkout -- .`) for fails.

System Interaction at a Glance

Local Cognitive Core

Spawns local LLM code generators to analyze JIRA tasks and write code patches.

Parallel Mutation

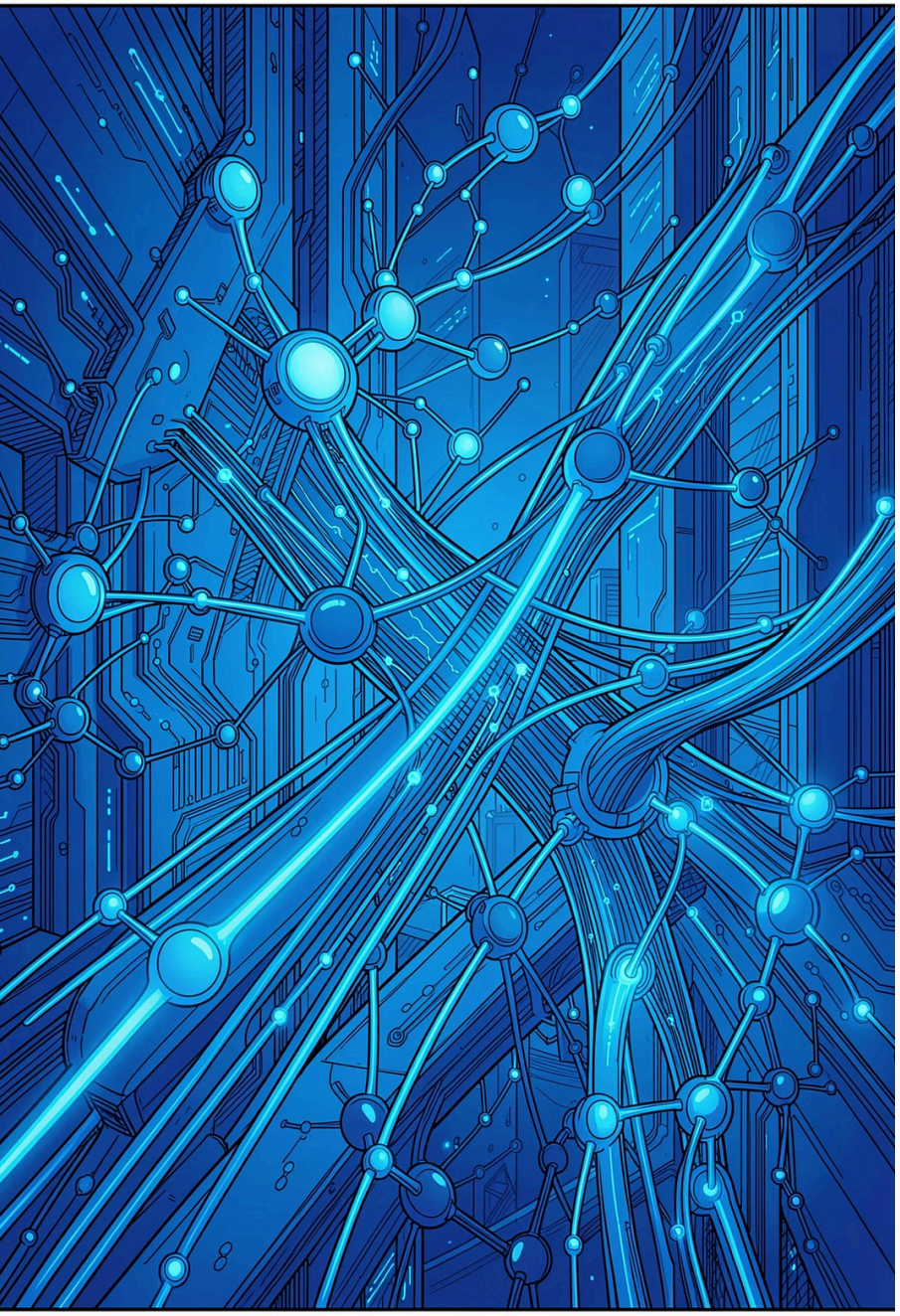
Generates three concurrent code mutants at varying temperatures to explore diverse logical approaches.

Adversarial Audit

The critic.py model independently evaluates code differences, test outputs, and compiler logs.

ANJANEYA Memory

Successful runs crystallized as vectors in a shared LanceDB Manifold for all agents to reuse.



Key Takeaways

Local & Autonomous

Runs entirely on local LLMs – no external API dependency.

Adversarial by Design

Every code change must pass a quantitative critic gate before commit.

Self-Correcting

AST Gas Meter and Goal Drift Guard enforce hard safety limits in real time.

Collective Intelligence

ANJANEYA ensures every success is remembered and reused by all agents.